

Master Autonomous Driving Data Specification (For LLM Prompting)

Comprehensive format guide for generating Risk Scoring & Trajectory Algorithms

1. System Architecture & Objective

Objective: Generate Python algorithms for Object Detection, Trajectory Prediction, and Collision Avoidance (AEB) using a multi-sensor autonomous driving setup.

Sensors Provided from Simulation:

1. Camera Raw Frames (.jpg images)
2. Camera Object Detections (Classifications & Bounding Boxes)
3. Radar Detections (Kinematic Physics Data)
4. LiDAR Point Clouds (.pcd files containing 3D spatial points)

The algorithms generated must accurately ingest these specific data formats to perform Sensor Fusion.

2. Data Format: Camera Raw Frames

Source Type: Standard JPEG image sequence (.jpg).

Resolution: Typically 1280x720 RGB.

Algorithm Requirements (Object Detection):

- The raw frames must be ingested by an optical inference model (e.g., YOLO11) to detect bounding boxes [x, y, w, h] and class confidences for autonomous driving classes (Car, Pedestrian, etc.).
- OpenCV (cv2) should be used for frame ingestion and preprocessing.

3. Data Format: Camera Object Detections

Source Type: Extracted CSV from MATLAB 'c_data.mat'.

Fields Available:

- Time: Simulation timestamp.
- SensorIndex: Camera ID.
- ObjectClassID: Integer representing the class (e.g., 1=Car).
- Measurement: 2D Bounding Box array [X_top_left, Y_top_left, Width, Height].

Algorithm Requirements (Ground Truth / Fusion):

- This data represents the simulation's built-in perception output. It should be used to validate the custom Object Detection model or fused with Radar to assign class identities to radar tracks.

Master Autonomous Driving Data Specification (For LLM Prompting)

Comprehensive format guide for generating Risk Scoring & Trajectory Algorithms

4. Data Format: Radar Kinematics

Source Type: Extracted CSV from MATLAB 'r_data.mat'.

Fields Available:

- Time: Simulation timestamp.
- SensorIndex: Radar ID.
- ObjectClassID: Integer representing the class (often generic for radar).
- Measurement: Numerical array containing physics metrics [Azimuth, Range, Range_Rate].

Definitions:

- Range (meters): Distance to the object.
- Azimuth (degrees/radians): Angle to the object relative to ego-vehicle.
- Range_Rate (m/s): Relative velocity of the object (negative = approaching, positive = receding).

Algorithm Requirements (Trajectory & Collision):

- Time-To-Collision (TTC) must be calculated dynamically: $TTC = Range / ABS(Range_Rate)$ (only if approaching).
- Constant Velocity (CV) Trajectory Prediction should use the Azimuth and Range_Rate to predict future X/Y coordinates of the object over a 1 to 3-second horizon.

5. Data Format: LiDAR Point Clouds

Source Type: Sequence of standard .pcd (Point Cloud Data) files.

Fields Available (Per Point):

- X, Y, Z: 3D spatial coordinates in meters relative to the ego-vehicle.

Algorithm Requirements (Clustering & Validation):

- Must use Python 'open3d' library to ingest .pcd files.
- Step 1: Apply RANSAC (Random Sample Consensus) to segment and remove the ground plane (the road).
- Step 2: Apply DBSCAN (Density-Based Spatial Clustering) on remaining points to group them into solid objects.
- Step 3: Compute Axis-Aligned Bounding Boxes (AABB) for each cluster to determine volumetric size and center (X_c, Y_c, Z_c).
- Step 4: Fuse this 3D centroid data with the Radar data to confirm physical presence and distance.

6. Instructions for Algorithm Formulation

When generating the Python scripts, the LLM MUST adhere to the following logic pipeline:

1. Perception Phase: Run YOLO on Camera JPEG frames to identify visual bounding boxes and classes.

Master Autonomous Driving Data Specification (For LLM Prompting)

Comprehensive format guide for generating Risk Scoring & Trajectory Algorithms

2. Clustering Phase: Process simultaneous LiDAR .pcd frames with DBSCAN to find 3D geometric clusters.
3. Sensor Fusion Phase: Match YOLO 2D bounding boxes and LiDAR 3D clusters with the incoming Radar measurements (using spatial heuristics) to assign a Range and Range_Rate to the detected objects.
4. Risk Scoring Phase: For each fused object, calculate TTC.
 - If $TTC < 1.5s$: Trigger Autonomous Emergency Braking (CRITICAL RISK).
 - If $TTC < 3.0s$: Trigger Driver Warning (HIGH RISK).
 - Else: Monitor (SAFE).
5. Trajectory Prediction: Implement a Kalman Filter or Constant Velocity model updating the predicted path of the object across sequential timestamps.